



为人类减负的 AI 编程总指挥

"不论是 Prompt 工程,还是 MCP、Skill、Multi-Agent、Harness Engineering,
它们都在做同一件事 —— 构建有限且正确的上下文。"

<https://github.com/Happenmass/Cliclaw>

项目

Cliclaw · 元编排式编程总指挥

提交日期

2026 年 4 月

开篇宣言:三个让我重写认知的洞察

2025 年下半年到 2026 年初,我亲历了一场密集的"工具学习":从 ChatGPT 时代的 Prompt 调教,到 Cursor 的内联补全,到 Claude Code、Codex 等命令行编程 Agent,再到 MCP 协议、Skill 系统、Multi-Agent、Harness Engineer。每一次范式更迭都让我以为自己掌握了新东西,但用得越深,我越确信它们在解决同一个底层问题。

Cliclaw 不是又一个编程 Agent。它是我把下面这三个洞察推到极致后,亲手沉淀出来的一个工程答案。

01 所有"AI 工程"的本质,都是构建有限且正确的上下文

这不是哲学,而是由 Transformer 自回归原理决定的工程铁律:下一个 Token 的预测概率,由当前上下文窗口内的全部内容共同决定。所以无论形式怎么换——Prompt 改写、System Message 注入、Tool Call 描述、RAG 检索增强、Skill 加载、Sub-Agent 隔离上下文——工程的核心始终是:把对此刻决策有用的、且只有有用的内容,放进有限的上下文窗口里。

多了,就稀释信号、增加成本、提升幻觉概率;少了,就让模型在残缺信息上瞎猜。

过期的、不相关的、重复的内容,都是噪声成本。整个工程化方向,都是在做"上下文"的取舍、压缩、组织、隔离。

02 领域内的客观知识,模型已经能自己决策;主观与约定,才需要人来判断

按我对当前(2026 年初)模型能力的判断:只要给定正确的上下文与流程,模型在绝大部分领域内的客观技术问题上已经具备自主决策能力——选哪种数据结构、用何种排序、如何重构这段函数,模型选得比平均水平的人类工程师好。

但主观的、约定性的、价值取向的问题——"我们团队偏好哪种风格"、"这次发布 先动哪部分"、"客户最在意的是稳定还是速度"——必须由人来锚定。

我注意到 Claude Code 的 `auto` 模型路由其实就是这种思想的一种初步体现:简单子任务用快模型,复杂决策路由到强模型。我想把这条路推得更远——在自然语言交互层实现自动决策与上下文路由,让我作为使用者只关心"我要什么"和"我倾向什么",其余的事都交给系统。

03 真正稀缺的、不会被快速替代的,是"跨项目认知负担"的解决方案

我观察到市面上的编程 Agent 工具(Claude Code、Cursor、Codex CLI 等)正在快速 垂直化: 它们越来越擅长"在一个仓库内,一次会话中做好一件编程任务"。这是好事,但 它们解决不了我作为开发者真正头疼的问题:

当我同时管理 5 个项目、并行驱动 3 个 Agent、横跨 2 周时间线的时候 —— 我自己的 认知带宽才是瓶颈。我会忘记上周让 Agent A 做的决策、记不清项目 B 的代码约定、 没法快速在多个 tmux 会话间切换上下文。

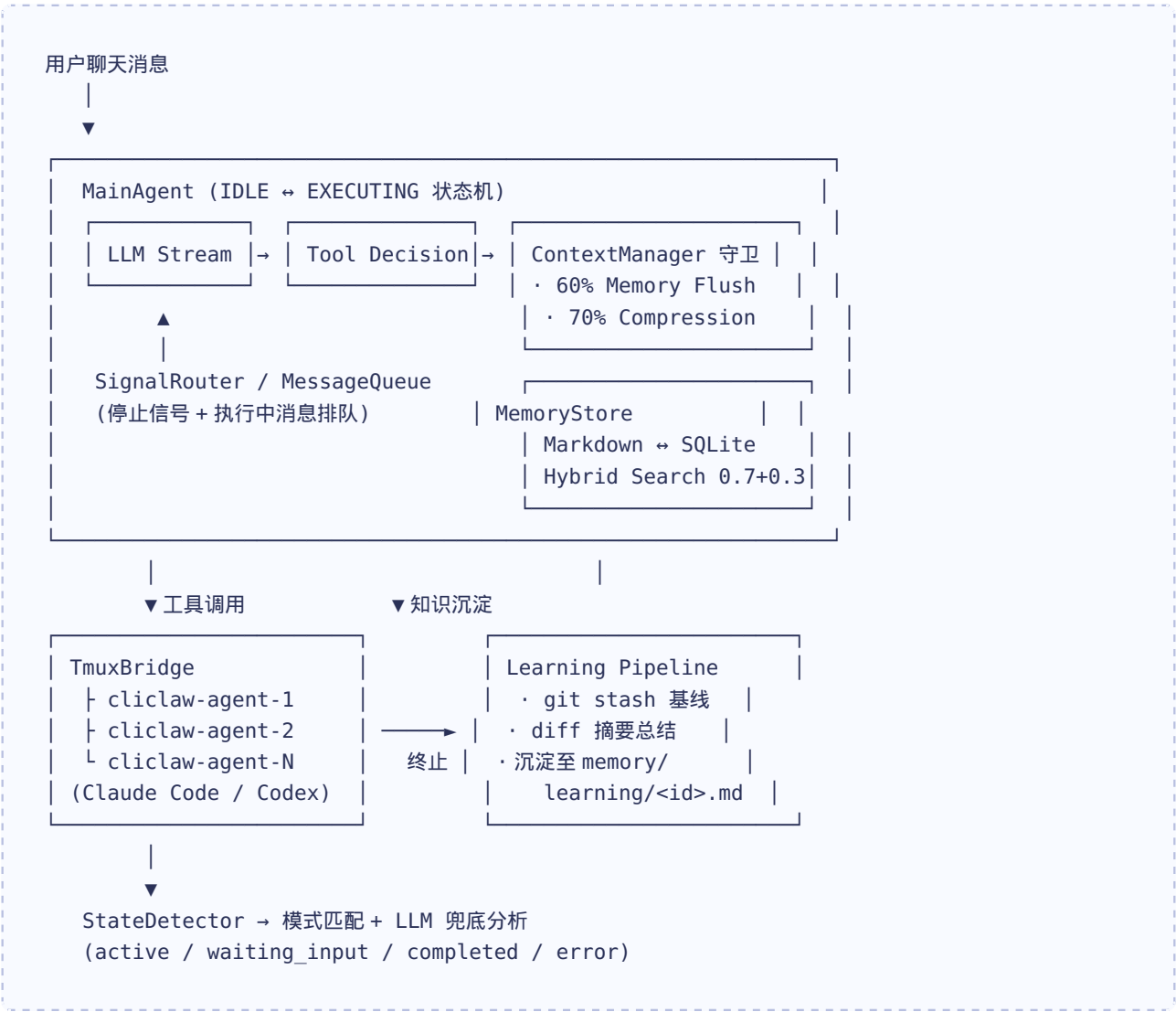
所以我把 Cliclaw 的护城河,放在了被忽视的层级: 跨项目的长期记忆、跨会话的 上下文隔离、自然语言的多 Agent 总指挥。这些能力的必要性不是我决定的,而是 自回归 **LLM** 的根本约束决定的 —— 只要 Token 预测仍依赖当前上下文, 就需要在 LLM 之外维护一套"人类记忆与意图的中间层"。

这三个洞察贯穿了我写下的每一行核心代码。下面的章节,我会用具体的 架构、模块和工程数字,把它们逐一兑现。

项目全景:Cliclaw 是什么

我把 **Cliclaw** 设计成一个基于聊天的元编排器(**meta-orchestrator**),通过 **tmux** 指挥诸如 **Claude Code**、**Codex** 这类专业编程 **Agent**。它以一个常驻 HTTP + WebSocket 服务的形式运行,自带一个 Web 聊天界面。我让 MainAgent 在一个 IDLE ↔ EXECUTING 的两态机里自主推进:接收自然语言意图、检索跨项目记忆、决策动用 哪些 tmux 编程 Agent、并在执行过程中持续做上下文管理。

核心数据流



我的三个支柱,呼应三大洞察



上下文工程

我用模块化 System Prompt、
双层阈值守卫、
Markdown↔SQLite 双存储、
Hybrid 搜索,把"有限且正确"做
成可复用的基础设施。



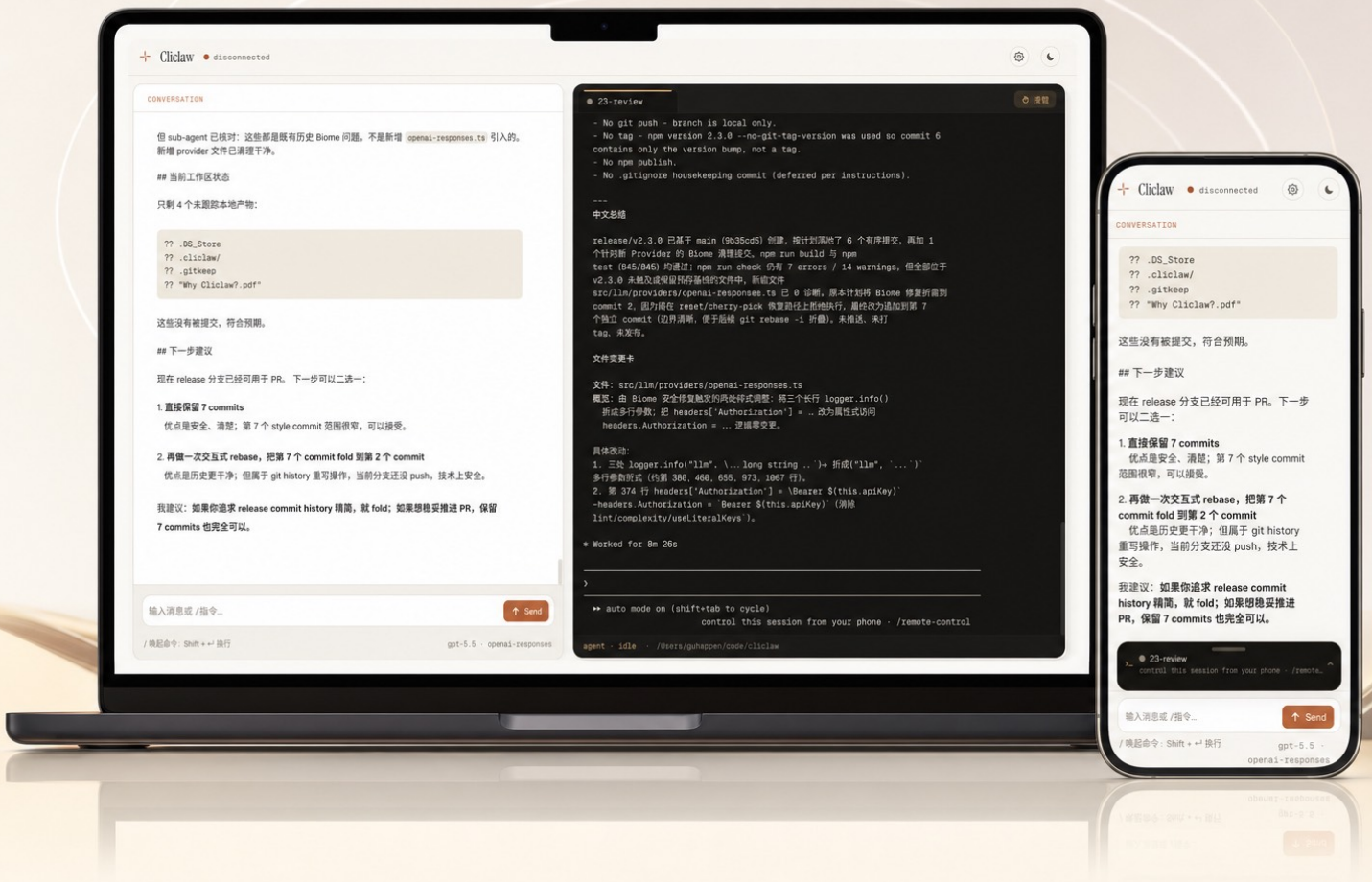
自主决策

我用两态状态机 + 18 个内置工
具,在自然语言层完成"理解意图
→ 路由 Agent → 监督执行 → 沉
淀学习"的全自动闭环。



长期记忆

我用跨项目 Persistent Memory
+ 子 Agent ChangeTracker 隔
离,来缓解我自己多项目并行时的
认知负担。



上下文工程,做成基础设施

如果"构建有限且正确的上下文"是 AI 工程的本质,那么把这件事做成基础设施 —— 让它 可观测、可干预、可复用 —— 就是我给 Cliclaw 定的第一性任务。

3.1 模块化 System Prompt

我让 `ContextManager` 把系统提示拆成可替换的模块,通过占位符 `{{compressed_history}}`、`{{memory}}`、`{{agent_capabilities}}` 在运行时注入。每一类内容(对话压缩快照 / 长期记忆 召回 / 当前可用 Agent 与 Skill 描述)都有独立的生命周期与刷新策略 —— 这让 Prompt 不再是一个不可变的字符串模板,而是一组可观测、可单独优化的"上下文流"。

3.2 双层阈值守卫:Flush 60% → Compress 70%

我观察到大多数 Agent 系统只有"压缩"一道防线,触发时机晚、信息损失大。所以我在 `ContextManager` 里设了两道闸,在每一轮工具执行间隙 自动检查:

```
// src/core/context-manager.ts (节选 · 简化)
if (estimateTokens() / contextWindow > 0.60) {
  // 第一道: Memory Flush — 不修改主对话, 独立 LLM 调用,
  // 把最近 30 条消息中"值得长期记的部分"通过 memory_edit 持久化
  await runMemoryFlush(recentMessages);
}
if (estimateTokens() / contextWindow > 0.70) {
  // 第二道: 历史压缩 — 调用 history-compressor prompt 生成摘要,
  // 清空对话, 通过 POST_COMPACTION_CONTEXT 重新注入语境
  await compressHistory();
}
```

为什么我要做两道? 因为压缩天然有损 —— 当上下文已经接近 70% 时再去 抢救,"什么是值得长期记忆的"这件事本身就难以判断。我在 60% 时先做一次纯粹的"知识抽取",让重要决策、用户偏好、约定性细节先溢出到记忆文件;再到 70% 时,压缩就只需 关心"对话连续性"这一件事。两道闸在职能上正交,损失最小化。

3.3 Hybrid Token 计数:精确与即时的折中

API 真实 token 数最准确,但只在 LLM 响应回来后才知,没法用来做"发送前"的预算决策。所以我用了 一个混合策略: `last_known_token_count` (API 实报值) + `pending_chars / 4` (尚未 API 化的字符

估计)。这个值在每次 LLM 响应后通过 `reportUsage()` 校准并持久化到 SQLite,使得 下一次发送前的阈值检查既精确又即时。

3.4 Memory:Markdown 是真理源,SQLite 是索引

这是我有意识做出的架构选择。LLM 的记忆体若只活在数据库里,迁移、备份、人工修订都很笨重;若只活在 Markdown 里,搜索召回又跟不上 Agent 实时决策的速度。我让两者各司其职:

- 真理源 = `~/.cliclawn/memory/**/*.md`,人类可读、可 git diff、可手编、可跨设备同步。
- 索引 = SQLite 的 `chunks_fts` (FTS5 关键词) + `chunks_vec_*` (`sqlite-vec` 向量 KNN)+ `embedding_cache` (避免重复嵌入)。索引随时可全量重建,嵌入模型变更会自动触发重新索引 (`hasModelChanged()` 检测)。

Hybrid 搜索:0.7 向量 + 0.3 BM25 + 时间衰减

纯向量召回失之过宽,纯关键字失之过窄。我写的 `mergeHybridResults()` 把两路结果加权合并,再叠加按"半衰期 30 天"配置的时间衰减,让近期日志类记忆与久远的 约定性记忆都能被合理排序。当向量后端不可用(无 API key、无网络)时,我让系统自动降级到 纯 FTS,保证可用性。

设计回报。这套上下文工程基础设施让我在长达数小时、跨数十个工具 调用、跨多个项目目录的会话里,依然能保持"对话连贯、记忆精准、决策稳定"——不是因为模型变强了,而是因为**我给模型的输入,始终是有限且正确的。**

从 Auto-Routing 到自然语言级自主决策

洞察二说: 客观知识模型自己能定,主观意图必须人来锚。要让这件事在自然语言交互层落地,我必须回答三个问题: 什么时候该停下来问人?什么时候可以自己往下推? 做错了怎么挽回?

4.1 IDLE ↔ EXECUTING:一个最小够用的状态机

我看到很多 Agent 框架走向了复杂的多状态有限自动机,自己反其道而行 —— 只用 两个状态。简化的代价是把"决策"压到工具层,但工具层的设计是可以被我严密 审查的。

当前态	触发	下一态	语义
IDLE	用户消息 → LLM 流式响应,只有文本	IDLE	纯对话,无副作用
IDLE	用户消息 → LLM 返回工具调用	EXECUTING	开始自主推进
EXECUTING	工具执行 → LLM 仅文本应答	IDLE	自然回到对话
EXECUTING	调用 <code>mark_failed</code> / <code>escalate_to_human</code>	IDLE	终态工具: 主动放弃或上交
EXECUTING	<code>SignalRouter.isStopRequested()</code>	IDLE	用户 <code>/stop</code> 强制中断

我让每一轮工具调用之间,EXECUTING 自循环都做四件事:检查停止信号 → 排空 MessageQueue(执行期间我新发的消息)→ 检查上下文阈值(60% / 70%)→ 进入下一轮 LLM。这意味着我可以在 Agent 自动推进时随时插话,系统会在恰当的时机把我的话融入,而不是 粗暴中断。

4.2 18 个内置工具:把决策面切成可以严密审查的格子

我把 MainAgent 的所有"行动"压缩进 18 个内置工具,按职能分四组:

- **Agent 总指挥(7):** `create_agent`、`send_to_agent`、`respond_to_agent`、`interrupt_agent`、`inspect_agent`、`list_agents`、`kill_agent` —— 所有面向 tmux 编程 Agent 的指令都收口在这里,UI 上的"agent_update"摘要也是从这层流出。
- **记忆与技能(5):** `memory_search`、`memory_get`、`memory_edit`、`persistent_memory` (全局/项目级 MEMORY.md 的结构化区块)、`read_skill` (按需读取 SKILL.md 全文)。
- **侦察与任务(3):** `exec_command` (只读 bash 侦察)、`list_agent_tasks`、其他状态查询。

- 终态(2): `mark_failed`、`escalate_to_human` —— 当模型判断"客观能力不足"或"主观判断必须由人做"时,我让它主动调用这两个工具回到 IDLE。这是洞察二的 代码化体现: 系统主动识别"哪些事不该自己做"。

4.3 Learning Sessions:让"做完了"自动变成"学到了"(开发中)

一个我自己最得意的细节: 每个 cliclaw-* tmux Agent 启动时,我让 `ChangeTracker` 捕获 git 基线 —— 干净工作树取 HEAD SHA;脏工作树用 `git stash create` 取一个游离的 tree SHA(不污染 **stash** 栈)。Agent 终止时,我会计算完整的 unified diff(包含 untracked 文件,尊重 .gitignore), 再加上整个交互过程中的 prompt 列表,送进 `learning-summarizer` 做摘要。

结果是一条 `LearningEntry`,我可以再对它做:合并(merge)、重生成(regenerate)、最终提取到 `memory(flushToMemory)`。这条流水线我把每一步都做成 错误隔离的 —— 任何环节失败都不会阻塞 agent 的正常 kill 路径。

"做完一件事 → 自动总结这件事 → 沉淀到长期记忆 → 下次决策时被 *hybrid* 搜索召回。" —— 这个闭环让客观经验持续积累,而主观决定始终 由人在自然语言层把控。

为什么 Cliclaw 不会被快速替代

5.1 定位差异:垂直专家 vs 元编排

维度	Claude Code 类垂直工具	Cliclaw
解决的问题	在一个项目内,一次会话中,把一件编程任务做好	跨多个项目,跨多次会话,把多件事并行推进且不让人崩溃
面对的瓶颈	模型本身的代码能力	人类的认知带宽 + LLM 的上下文窗口
演化方向	越来越垂直 —— 编程交互、代码理解、调试体验持续打磨	越来越元 —— 跨项目记忆、多 Agent 编排、自然语言决策
替代关系	互补,不竞争。我通过 Adapter 层(claude-code、codex ...)把这些垂直工具当作"专科医生"去调用。	

5.2 不会被快速吃掉的三层护城河

护城河 ①:跨项目长期记忆,是产品形态背后的物理约束

我反复想过这件事: 只要 LLM 还是自回归的,Token 预测就只能依赖当前上下文窗口。这意味着 —— "我两周前在另一个项目里给 Agent 立的规矩"不可能自动出现在这次会话的预测 概率里,除非有一层外部记忆系统在恰当的时机主动把它召回并注入。这不是某个 产品功能,而是 Transformer 架构留给我们的结构性空缺。

所以我在 Cliclaw 里设计了 `persistent_memory` 工具(分 `user_profile`、`project_conventions`、`key_decisions`、`people_and_context`、`active_notes` 五个结构化区块) + Markdown 真理源 + Hybrid 搜索召回,用来填补这个空缺。我相信下一代 **LLM** 即使把 上下文窗口扩到百万级,也不能消除"跨项目"和"跨会话"的边界 —— 因为成本与噪声会先压垮它。

护城河 ②:上下文隔离 = **LLM** 时代的"进程隔离"

我让每个 cliclaw-* sub-agent 拥有独立的 ChangeTracker、独立的 prompt 累积、独立的 learning entry。MainAgent 不会"看到" sub-agent 内部的全部对话,只通过 `inspect_agent` /

`respond_to_agent` 这类窄接口交互。我把这种"上下文进程化"的隔离当作多 Agent 协同必经的工程范式——就像没有进程隔离的操作系统跑不稳一样,没有上下文隔离的多 Agent 系统也不可能被人信任。

护城河 ③:Skill 系统的"领域知识联邦"

我让 Skill 通过 `discoverSkills()` 从 Adapter 与工作区分别发现,工作区可以覆盖 **Adapter** 默认。团队不需要修改我维护的 Cliclaw 本体,就可以把自己的领域知识(代码规范、合规要求、内部 API 模式)注入主 Agent 的工具集与 Prompt。我希望它能让 Cliclaw 从"通用工具"长成"组织级可定制的工具",护城河会随团队定制的深度自然加深。

5.3 关于"会被替代"的诚实回答

会的。任何具体功能都会被替代。但我相信被替代的不会是"跨项目认知卸载"这个使命,而是我当前对它的**具体实现**。只要 LLM 还在自回归世界里、人还要同时管多个项目、上下文窗口还有上限,这个层级的工具就会一直存在。我要做的,是不断把这层做得更好,而不是去和垂直编程 Agent 抢地盘。

工程数据与未来路线

6.1 当前工程量

11,910

TypeScript 源代码行数

77

主要 .ts 源文件

58

单元/集成测试文件

18

MainAgent 内置工具

14

支持的 LLM 提供商

2

Agent Adapter (claude-code / codex)

6.2 技术栈与运行模式

- 语言与构建: TypeScript strict 模式、ES2022 target、Node16 模块解析、Biome 格式化、Vitest 测试。
- 持久化: SQLite(WAL 模式) 用于聊天历史、上下文状态、记忆索引、Learning 条目; `sqlite-vec` 提供向量 KNN;FTS5 提供关键字搜索。
- 嵌入模型: 自动回退链 —— OpenAI / Gemini / Voyage / Mistral / 本地 Qwen3-Embedding(基于 node-llama-cpp)/ none。模型变更自动触发全量重新索引。
- 运行形态: 单进程 HTTP + WebSocket 服务,默认端口 3120,Web 聊天 UI (`web/`) 提供流式响应、Slash 命令、实时 Agent 终端镜像。
- **i18n**: 内建 zh-CN / en-US 双语,LLM 提示中通过 `{{language_instruction}}` 注入语言指令。

6.3 我接下来 6 个月想做的事

- 更精细的 **Memory Tidy**: 我打算引入"知识置信度衰减"模型,让长期 未被召回的记忆自动归档,而不是只在阈值触发时才被处理。
- 多模态上下文: 我想把架构图、UI 截图、设计稿纳入 Memory,把 Hybrid 搜索从两路扩到向量 + 关键字 + 视觉特征三路融合。

- **Agent Adapter** 生态: 我会把 Adapter 层标准化为 npm 包,允许第 三方接入 Aider、Continue、Roo Code 等更多专业工具。
- 多人协作: 在保持上下文隔离的前提下,我想让团队成员共享部分 `persistent_memory` 区块,做团队级"组织记忆"。
- 更激进的自然语言路由: 我希望把"该用强模型还是快模型"、"该新建 Agent 还是复用"这类决策从工具层进一步下放到 router,让自然语言意图成为 唯一接口。

Cliclaw — 有限且正确的上下文,是 AI 工程的全部。